

重庆邮电大学

学生实验实习报告册

学年学期： 2025-2026 学年（1）学期

课程名称： 通信软件开发与应用

学生学院： 通信与信息工程学院

专业班级： 01042301

学生学号： 2023211281

学生姓名： 丁同勳

联系电话： 18019582857

重庆邮电大学教务处制

课程名称	通信软件开发与应用			课程编号	A2012230		
实验地点	YF315			实验时间	13 周 9-11 节		
校外指导教师	无			校内指导教师	梁燕		
实验名称	C/S 程序设计与实现						
评阅人签字		操作成绩 80%		报告成绩 20%		总评成绩	

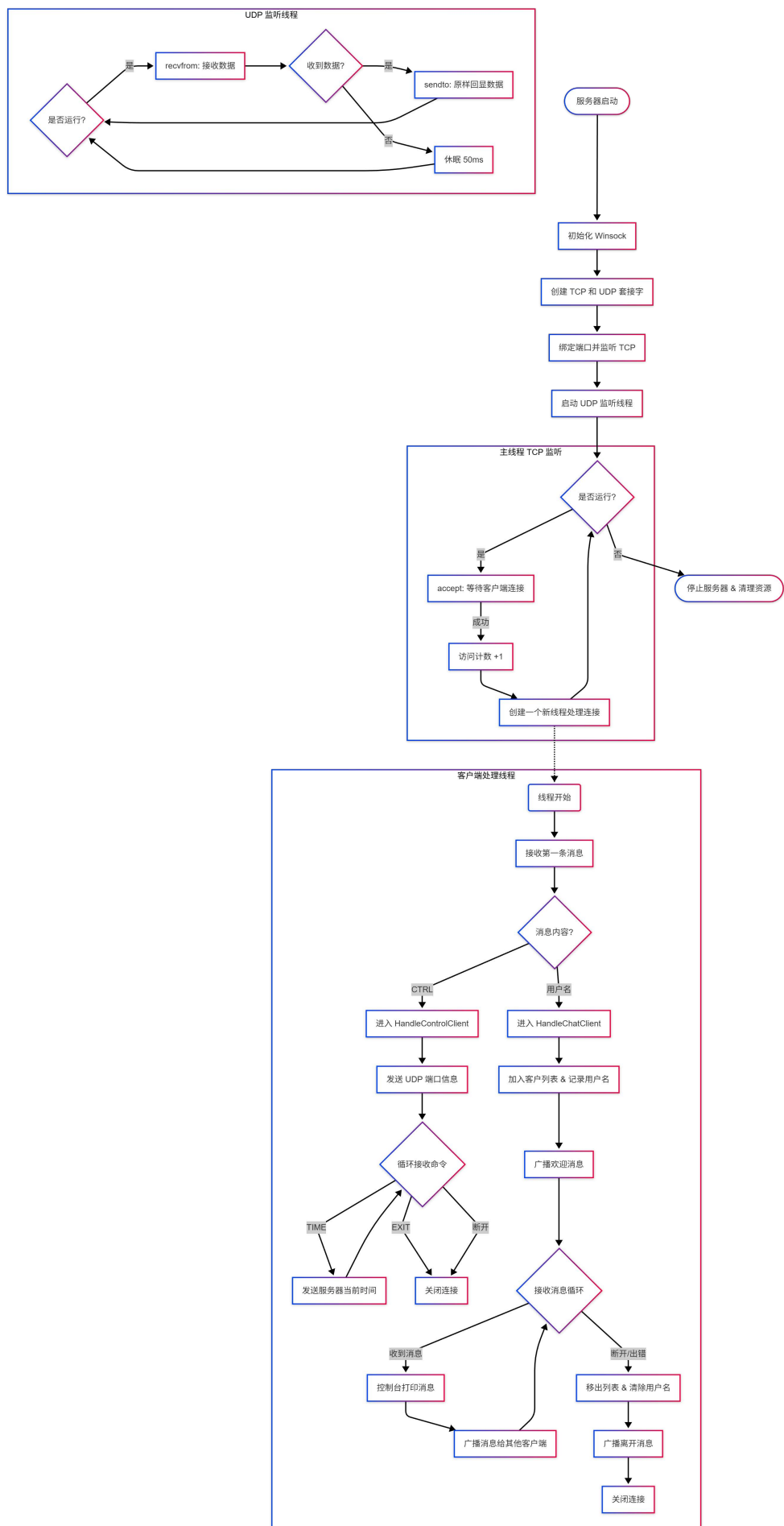
一、实验目的

- 1、掌握 C/S 程序基本架构
- 2、掌握通信程序中五参的基本概念
- 3、掌握基于 TCP、UDP 的 C/S 程序设计方法
- 4、掌握线程的基本概念
- 5、掌握一客户端一线程模式设计方法
- 6、初步掌握自定义协议方法

二、实验思路

服务端流程描述：

1. **启动与初始化：**解析命令行参数（IP、TCP 端口、UDP 端口），初始化 Winsock，创建并绑定 TCP 和 UDP 套接字。
2. **UDP 服务：**启动一个独立的线程 UdpListener，循环接收 UDP 数据包并原样发回（Echo 服务）。
3. **主循环：**主线程进入 while(running) 循环，调用 accept 阻塞等待客户端连接。
4. **连接分发：**当有新连接时，增加访问计数，并创建一个新的分离线程 HandleTCPConnection 来处理该套接字。
5. **协议识别：**新线程首先接收一次数据：
 - 如果收到 "CTRL"：判定为控制客户端。服务器发送 UDP 端口号，随后进入循环处理 "TIME"（返回时间）或 "EXIT" 指令。
 - 如果收到 其他内容：判定为聊天客户端，将内容作为用户名。
6. **聊天处理：**
 - 将客户端 Socket 和用户名存入受互斥锁保护的容器中。
 - 广播“欢迎进入”消息。
 - 循环接收客户端消息并广播给其他所有在线用户。
 - 连接断开时，从列表移除并广播“离开”消息。



客户端流程描述:

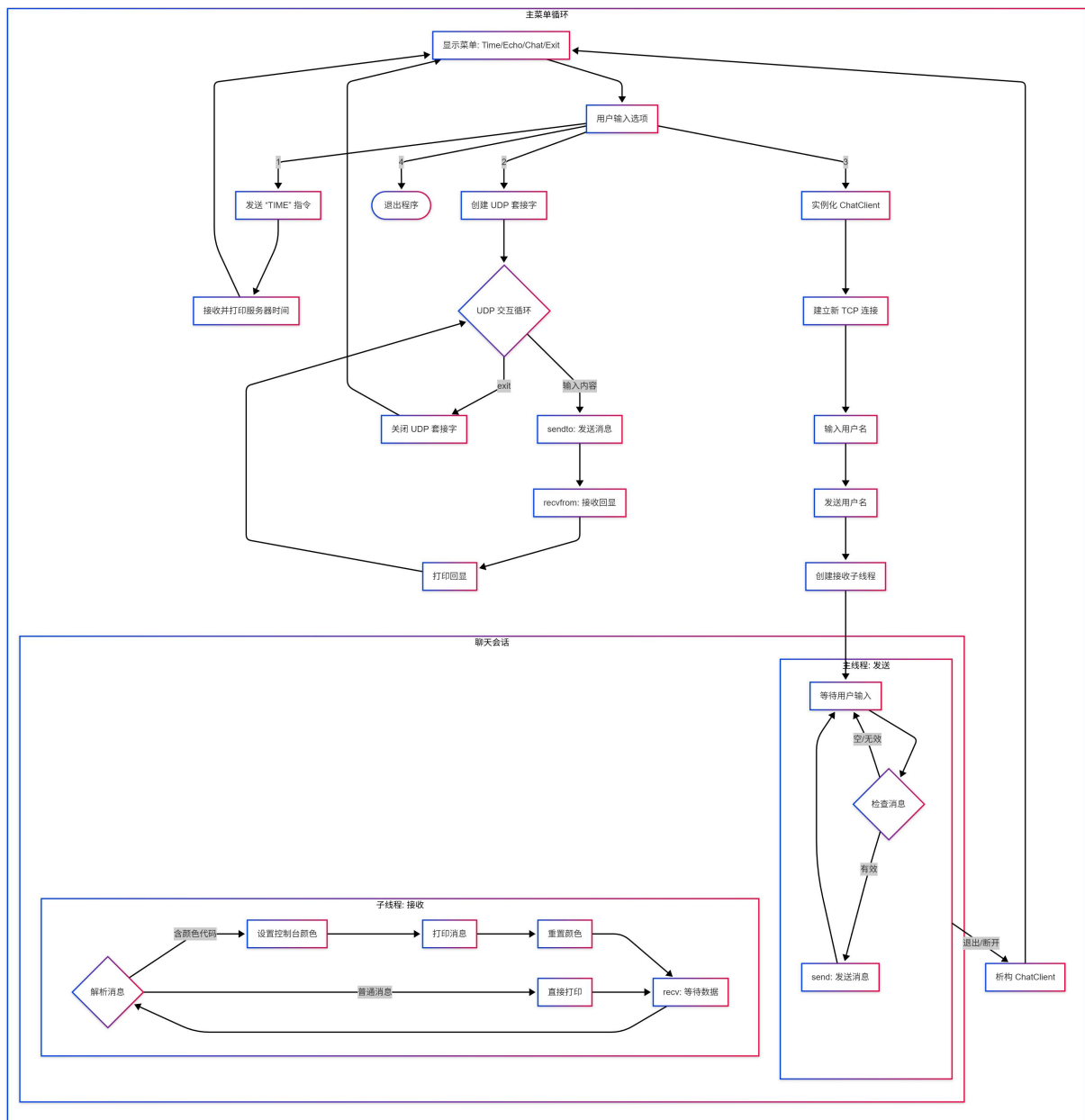
1. 初始化与控制连接:

- 启动时连接服务器 TCP 端口, 发送 "CTRL".
- 获取服务器返回的 UDP 端口号 (用于后续的 Echo 功能)。

2. 主菜单循环:

- **选项 1 (Time):** 利用已建立的控制 Socket 发送 "TIME", 接收并显示服务器时间。
- **选项 2 (Echo):** 创建一个新的 UDP Socket。用户输入消息后发送给服务器 (使用初始化时获取的 UDP 端口), 接收服务器的回显并打印, 输入 "exit" 退出此模式。
- **选项 3 (Chat):** 创建一个 ChatClient 对象。
 - 建立一个新的 TCP 连接到服务器。
 - 提示用户输入用户名并发送。
 - **创建接收线程:** 专门负责 recv 服务器消息。如果消息包含特殊格式 (如 #红色), 则解析颜色代码, 修改控制台字体颜色后打印, 然后重置颜色。
 - **主线程发送循环:** 在 Start() 中循环读取用户输入 (std::getline) 并发送给服务器。
- **选项 4 (Exit):** 退出程序。





三、源代码

```

=====Server.cpp=====
#include <ws2tcpip.h>
#include <winsock2.h>
#include <iostream>
#include <vector>
#include <mutex>
#include <thread>
#include <algorithm>
#include <csignal>
#include <map>
#include <string>
#include <chrono>
#include <ctime>
//2023211281-丁同勛-Server

```

```

using namespace std;

#pragma comment(lib, "ws2_32.lib")

class ChatServer {
public:
    ChatServer(const string& IP, int port, int udpPort);
    void Start();
    void Stop();
    static void SignalHandler(int signal);
    static ChatServer* serverInstance;

private:
    void HandleTCPConnection(SOCKET clientSocket);
    void HandleChatClient(SOCKET clientSocket, const string& preReceivedUsername);
    void HandleControlClient(SOCKET clientSocket);
    void BroadcastMessage(const char* message, SOCKET senderSocket, int type);
    void PrintError(const string& message);
    void UdpListener();

    vector<SOCKET> clients; // 存储客户端套接字
    map<SOCKET, string> clientUsernames; // 存储客户端用户名
    mutex clientsMutex; // 保护对客户端的访问
    SOCKET serverSocket;
    SOCKET udpSocket;
    bool running = false;
    int accessCount = 0;
    int udpPort;
    string ip;
};

ChatServer* ChatServer::serverInstance = nullptr;

ChatServer::ChatServer(const string& IP, int port, int udpPort) : udpPort(udpPort), ip(IP) {
    WSADATA wd;
    if (WSAStartup(MAKEWORD(2, 2), &wd) != 0) {
        PrintError("WSAStartup失败");
        exit(1);
    }

    serverSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (serverSocket == INVALID_SOCKET) {
        PrintError("创建套接字失败");
        exit(1);
    }
}

```

```

sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(port);
inet_pton(AF_INET, IP.c_str(), &addr.sin_addr);

if (bind(serverSocket, (SOCKADDR*)&addr, sizeof(addr)) == SOCKET_ERROR) {
    PrintError("绑定失败");
    exit(1);
}

if (listen(serverSocket, SOMAXCONN) == SOCKET_ERROR) {
    PrintError("监听失败");
    exit(1);
}

// 创建并绑定UDP套接字
udpSocket = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
if (udpSocket == INVALID_SOCKET) {
    PrintError("创建UDP套接字失败");
    exit(1);
}
sockaddr_in udpAddr;
udpAddr.sin_family = AF_INET;
udpAddr.sin_port = htons(udpPort);
inet_pton(AF_INET, IP.c_str(), &udpAddr.sin_addr);

if (bind(udpSocket, (SOCKADDR*)&udpAddr, sizeof(udpAddr)) == SOCKET_ERROR) {
    PrintError("UDP绑定失败");
    closesocket(udpSocket);
    exit(1);
}

cout << "服务器正在监听 TCP 端口 " << port << " 和 UDP 端口 " << udpPort << "... " << endl;
running = true;

// 启动UDP监听线程
thread(&ChatServer::UdpListener, this).detach();
}

void ChatServer::Start() {
    while (running) {
        SOCKET clientSocket = accept(serverSocket, nullptr, nullptr);
        if (clientSocket != INVALID_SOCKET) {
            {
                lock_guard<mutex> lock(clientsMutex);
            }
        }
    }
}

```

```

        accessCount++; // 增加访问计数
        cout << "当前访问次数: " << accessCount << endl;
    }
    thread(&ChatServer::HandleTCPConnection, this, clientSocket).detach();
}
}

void ChatServer::HandleTCPConnection(SOCKET clientSocket) {
    // 首先接收一段消息来判断连接类型（控制或聊天）。
    char buf[256] = {0};
    int ret = recv(clientSocket, buf, sizeof(buf) - 1, 0);
    if (ret <= 0) {
        closesocket(clientSocket);
        return;
    }

    string received(buf, ret);
    if (received == "CTRL") {
        HandleControlClient(clientSocket);
    } else {
        // 把接收到的数据当作用户名，继续聊天处理
        HandleChatClient(clientSocket, received);
    }
}

void ChatServer::HandleChatClient(SOCKET clientSocket, const string& preReceivedUsername) {
    string username = preReceivedUsername;
    if (username.empty()) {
        char usernameBuf[100] = {0};
        int r = recv(clientSocket, usernameBuf, sizeof(usernameBuf) - 1, 0);
        if (r <= 0) {
            closesocket(clientSocket);
            return;
        }
        username = string(usernameBuf, r);
    }

    {
        lock_guard<mutex> lock(clientsMutex);
        clients.push_back(clientSocket);
        clientUsernames[clientSocket] = username; // 存储用户名
    }

    cout << "欢迎 " << username << " 进入聊天室! " << endl;
}

```



```

char welcomeMessage[200];
snprintf(welcomeMessage, sizeof(welcomeMessage), "欢迎 %s 进入聊天室!", username.c_str());
send(clientSocket, welcomeMessage, (int)strlen(welcomeMessage) + 1, 0);
BroadcastMessage(welcomeMessage, clientSocket, 0);

char bufMsg[200] = {0};
int ret;

do {
    memset(bufMsg, 0, sizeof(bufMsg));
    ret = recv(clientSocket, bufMsg, sizeof(bufMsg) - 1, 0);

    if (ret > 0) {
        cout << username << " 说: " << bufMsg << endl;
        BroadcastMessage(bufMsg, clientSocket, 1);
    }
} while (ret > 0);

{
    lock_guard<mutex> lock(clientsMutex);
    clients.erase(remove(clients.begin(), clients.end(), clientSocket), clients.end());
    clientUsernames.erase(clientSocket); // 移除用户名
}

cout << username << " 离开了聊天室" << endl;
BroadcastMessage(username.c_str(), clientSocket, 2);
closesocket(clientSocket);
}

void ChatServer::HandleControlClient(SOCKET clientSocket) {
    // 发送UDP端口和START命令
    char info[128];
    snprintf(info, sizeof(info), "UDPPORT:%d;START", udpPort);
    send(clientSocket, info, (int)strlen(info) + 1, 0);

    // 进入控制命令循环
    char buf[128];
    while (true) {
        memset(buf, 0, sizeof(buf));
        int ret = recv(clientSocket, buf, sizeof(buf) - 1, 0);
        if (ret <= 0) break;
        string cmd(buf, ret);
        if (cmd == "TIME") {
            // 发送当前时间
            auto now = chrono::system_clock::now();
            time_t t = chrono::system_clock::to_time_t(now);

```

```

        char timestr[64] = {0};
        if (ctime_s(timestr, sizeof(timestr), &t) == 0) {
            // ctime_s 返回带换行的字符串
            send(clientSocket, timestr, (int)strlen(timestr) + 1, 0);
            cout << "已发送时间给控制客户端: " << timestr << endl;
        } else {
            string err = "Failed to get time";
            send(clientSocket, err.c_str(), (int)err.size() + 1, 0);
            cout << "发送时间失败" << endl;
        }
    } else if (cmd == "EXIT") {
        break;
    } else {
        // 未知命令, 回显
        string r = string("Unknown command: ") + cmd;
        send(clientSocket, r.c_str(), (int)r.size() + 1, 0);
        cout << "收到未知命令: " << cmd << endl;
    }
}

closesocket(clientSocket);
}

void ChatServer::UdpListener() {
    // 简单的回显服务器: 接收到什么就发送回去
    sockaddr_in fromAddr;
    int fromLen = sizeof(fromAddr);
    char buf[1024];
    while (running) {
        memset(&fromAddr, 0, sizeof(fromAddr));
        memset(buf, 0, sizeof(buf));
        int r = recvfrom(udpSocket, buf, (int)sizeof(buf) - 1, 0, (SOCKADDR*)&fromAddr, &fromLen);
        if (r > 0) {
            // 回显
            sendto(udpSocket, buf, r, 0, (SOCKADDR*)&fromAddr, fromLen);
            cout << "UDP回显: " << string(buf, r) << endl;
        }
        else {
            this_thread::sleep_for(chrono::milliseconds(50));
        }
    }
}

void ChatServer::BroadcastMessage(const char* message, SOCKET senderSocket, int type) {
    lock_guard<mutex> lock(clientsMutex);
    for (const SOCKET& otherClient : clients) {

```

```

        if (otherClient != senderSocket) {
            char msg[200];
            if (type == 1) {
                snprintf(msg, sizeof(msg), "%s 说: %s", clientUsernames[senderSocket].c_str(),
message);
            }
            else if (type == 2) {
                snprintf(msg, sizeof(msg), "服务器消息: %s 离开了聊天室", message);
            }
            else {
                snprintf(msg, sizeof(msg), "服务器消息: 新成员加入! %s", message);
            }
            send(otherClient, msg, (int)strlen(msg) + 1, 0);
        }
    }
}

void ChatServer::PrintError(const string& message) {
    cout << message << ": " << GetLastError() << endl;
}

void ChatServer::Stop() {
    running = false; // 设置标志为false, 退出主循环
    closesocket(serverSocket); // 关闭监听套接字
    closesocket(udpSocket);
    WSACleanup(); // 清理Winsock
    cout << "服务器已关闭。" << endl;
}

// 信号处理函数
void ChatServer::SignalHandler(int signal) {
    if (serverInstance) {
        serverInstance->Stop();
    }
}

int main(int argc, char* argv[]) {

    string IP = "127.0.0.1";
    int PORT = 3000;
    int UDPPORT = 4001; // 默认 UDP 端口, 用户可以修改

    if (argc == 1) {
        cout << "未设置指定 IP 和端口号, 默认使用 127.0.0.1:3000, UDP端口 " << UDPPORT << " " << endl;
    }
    else if (argc == 2) {

```

```

        cout << "IP : " << argv[1] << ", 未设置指定端口号, 默认使用 3000, UDP端口 " << UDPPORT <<
"" << endl;
        IP = argv[1];
    }
    else if (argc == 3) {
        cout << "正在连接: " << argv[1] << ":" << argv[2] << endl;
        IP = argv[1];
        PORT = std::stoi(argv[2]); // 处理char* 转 int
    }
    else if (argc >= 4) {
        IP = argv[1];
        PORT = std::stoi(argv[2]);
        UDPPORT = std::stoi(argv[3]);
        cout << "使用: " << IP << ":" << PORT << " UDP:" << UDPPORT << endl;
    }

    ChatServer server(IP, PORT, UDPPORT);
    ChatServer::serverInstance = &server;

    // 设置信号处理
    signal(SIGINT, ChatServer::SignalHandler);
    signal(SIGTERM, ChatServer::SignalHandler);

    server.Start();

    return 0;
}

```

```

=====Client.cpp=====
#include <winsock2.h>
#include <WS2tcpip.h>
#include <iostream>
#include <thread>
#include <memory>
#include <string>
#include <windows.h> // 用于改变控制台颜色
#include <chrono>
//2023211281-丁同勛-Client

using namespace std;

#pragma comment(lib, "ws2_32.lib")

class ChatClient {
public:

```

```

    ChatClient(const string& ipAddress, int port);
    ~ChatClient();
    void Start();

private:
    static DWORD WINAPI RecvMessage(LPVOID lpThread);
    void SendUsername();
    void SendMessage();
    static void SetConsoleColor(const string& colorName); // 设置控制台颜色为静态函数
    static void ResetConsoleColor(); // 重置控制台颜色为静态函数

    SOCKET socket_;
    bool running_;
};

// 新增控制/功能客户端
class ControlClient {
public:
    ControlClient(const string& ipAddress, int port);
    ~ControlClient();
    // 获取UDP端口并发送START
    int RequestUdpPort();
    string SendTimeRequest();
private:
    SOCKET ctrlSocket;
};

ChatClient::ChatClient(const string& ipAddress, int port) : running_(true) {
    WSADATA wd;
    if (WSAStartup(MAKEWORD(2, 2), &wd) != 0) {
        throw runtime_error("WSAStartup error: " + to_string(GetLastError()));
    }

    socket_ = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if (socket_ == INVALID_SOCKET) {
        throw runtime_error("Socket error: " + to_string(GetLastError()));
    }

    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    inet_pton(AF_INET, ipAddress.c_str(), &addr.sin_addr);

    if (connect(socket_, (SOCKADDR*)&addr, sizeof(addr)) == SOCKET_ERROR) {
        throw runtime_error("Connect error: " + to_string(GetLastError()));
    }
}

```

```

// 发送用户名作为第一条消息
SendUsername(); // 发送用户名

// 启动接收线程
HANDLE hThread = CreateThread(NULL, 0, RecvMessage, (LPVOID)socket_, 0, NULL);
if (hThread == NULL) {
    throw runtime_error("创建接收线程失败: " + to_string(GetLastError()));
}
CloseHandle(hThread);
}

ChatClient::~ChatClient() {
    running_ = false; // 设置为 false, 以便停止接收消息
    closesocket(socket_); // 关闭套接字
    WSACleanup(); // 清理 Winsock
}

void ChatClient::Start() {
    SendMessage(); // 发送聊天消息
}

void ChatClient::SendUsername() {
    std::string username;
    int space = 1;
    while (space) { // 检查用户名是否有空格
        std::cout << "请输入您的用户名: ";
        std::getline(std::cin, username);
        if (username.find(' ') == string::npos) {
            space = 0;
        }
        else {
            cout << "用户名不能含有空格, 请重新输入!" << endl;
        }
    }
}

// 发送用户名到服务器
if (send(socket_, username.c_str(), username.size(), 0) == SOCKET_ERROR) {
    std::cout << "发送用户名失败: " << GetLastError() << std::endl;
    return;
}

// 接收服务器的确认消息
char buf[100] = { 0 };
recv(socket_, buf, sizeof(buf), 0);
std::cout << "服务器确认: " << buf << std::endl;

```

```

    std::cout << "Tip: #颜色 [消息] 可以改变消息颜色。支持的颜色有红色、绿色、蓝色、黄色、洋红和青色。\\n" << std::endl;
}

void ChatClient::SendMessage() {
    std::string message;
    while (running_) {
        cout << "请输入聊天内容: ";
        std::getline(std::cin, message); // 使用 getline 来处理包含空格的输入

        if (!message.empty()) { // 确保不发送空消息

            size_t spacePos = string::npos;
            spacePos = message.find(' '); // 获取颜色后的空格位置
            if (message[spacePos + 1] != '\\0') { // 检查含颜色的输入是否为空
                if (send(socket_, message.c_str(), message.size(), 0) == SOCKET_ERROR) {
                    cout << "发送消息失败: " << GetLastError() << endl;
                    break;
                }
                else {
                    cout << "消息发送成功!" << endl;
                }
            }
            else {
                cout << "消息不能为空, 请重新输入。" << endl;
            }
        }
        else {
            cout << "消息不能为空, 请重新输入。" << endl;
        }
    }
}

DWORD WINAPI ChatClient::RecvMessage(LPVOID lpThread) {
    SOCKET s = (SOCKET)lpThread;
    char buf[100] = { 0 };
    int ret = 0;

    while (true) {
        ret = recv(s, buf, sizeof(buf), 0);

        if (ret > 0) {
            string receivedMessage(buf, ret);

            // 清除未使用的输入提示并打印消息
            cout << "\\033[1K\\r";

```

```

// 提取颜色和消息内容
int count = 0;
size_t spacePos1 = string::npos;
size_t spacePos2 = string::npos;
size_t end = string::npos;
size_t startPos = 0;

spacePos1 = receivedMessage.find(' ', startPos); // 获取用户名后的空格位置
startPos = spacePos1 + 1; // 更新查找位置, 防止重复查找

spacePos2 = receivedMessage.find(' ', startPos); // 获取消息前的空格位置
startPos = spacePos2 + 1; // 更新查找位置, 防止重复查找

end = receivedMessage.find('\0', startPos); // 获取消息结尾

if (receivedMessage.size() > 5 && receivedMessage[spacePos1 + 5] == '#' && spacePos2 !=
string::npos) {
    string name = receivedMessage.substr(0, spacePos1 + 5); // 提取发送者用户名
    string colorCode = receivedMessage.substr(spacePos1 + 5, 5); // 识别颜色
    string msgContent = receivedMessage.substr(spacePos2 + 1, end - spacePos2 + 1); //
提取消息内容

    // 设置颜色
    SetConsoleColor(colorCode);
    cout << name;
    cout << msgContent << endl;

    // 重置颜色
    ResetConsoleColor();
}
else {
    cout << buf << endl;
}

cout << "请输入聊天内容: "; // 提示用户输入

}
else {
    cout << "接收消息失败: " << GetLastError() << endl;
    break; // 连接关闭或出错
}
}
return 0;
}

```



```

void ChatClient::SetConsoleColor(const string& colorCode) {
    // 根据颜色代码设置控制台颜色
    if (colorCode == "#红色") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_RED |
FOREGROUND_INTENSITY);
    }
    else if (colorCode == "#绿色") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_GREEN |
FOREGROUND_INTENSITY);
    }
    else if (colorCode == "#蓝色") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_BLUE |
FOREGROUND_INTENSITY);
    }
    else if (colorCode == "#黄色") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_RED |
FOREGROUND_GREEN | FOREGROUND_INTENSITY);
    }
    else if (colorCode == "#洋红") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_RED |
FOREGROUND_BLUE | FOREGROUND_INTENSITY);
    }
    else if (colorCode == "#青色") {
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_GREEN |
FOREGROUND_BLUE | FOREGROUND_INTENSITY);
    }
    else {
        // 默认颜色
        SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_RED |
FOREGROUND_GREEN | FOREGROUND_BLUE);
    }
}

void ChatClient::ResetConsoleColor() {
    // 重置为默认颜色
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_RED | FOREGROUND_GREEN |
FOREGROUND_BLUE);
}

ControlClient::ControlClient(const string& ipAddress, int port) {
    WSADATA wd;
    if (WSAStartup(MAKEWORD(2, 2), &wd) != 0) {
        throw runtime_error("WSAStartup error: " + to_string(GetLastError()));
    }

    ctrlSocket = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
}

```

```

    if (ctrlSocket == INVALID_SOCKET) {
        throw runtime_error("Socket error: " + to_string(GetLastError()));
    }

    sockaddr_in addr;
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    inet_pton(AF_INET, ipAddress.c_str(), &addr.sin_addr);

    if (connect(ctrlSocket, (SOCKADDR*)&addr, sizeof(addr)) == SOCKET_ERROR) {
        throw runtime_error("Connect error: " + to_string(GetLastError()));
    }

    // 表明控制连接
    send(ctrlSocket, "CTRL", 4, 0);
}

ControlClient::~ControlClient() {
    closesocket(ctrlSocket);
}

int ControlClient::RequestUdpPort() {
    char buf[256] = { 0 };
    int r = recv(ctrlSocket, buf, sizeof(buf) - 1, 0);
    if (r <= 0) return -1;
    string s(buf, r);
    // 格式 UDPPORT:<port>;START
    size_t p = s.find("UDPPORT:");
    if (p == string::npos) return -1;
    size_t semi = s.find(';', p);
    string num = s.substr(p + 8, semi - (p + 8));
    return stoi(num);
}

string ControlClient::SendTimeRequest() {
    send(ctrlSocket, "TIME", 4, 0);
    char buf[256] = { 0 };
    int r = recv(ctrlSocket, buf, sizeof(buf) - 1, 0);
    if (r <= 0) return string();
    return string(buf, r);
}

int main(int argc, char* argv[]) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), FOREGROUND_RED | FOREGROUND_GREEN |
    FOREGROUND_BLUE);

```

```

string IP = "127.0.0.1";
int PORT = 3000;

if (argc == 1) {
    cout << "未设置指定 IP 和端口号，默认使用 127.0.0.1:3000" << endl;
}
else if (argc == 2) {
    cout << "IP : " << argv[1] << ", 未设置指定端口号，默认使用 3000" << endl;
    IP = argv[1];
}
else if (argc == 3) {
    cout << "正在连接: " << argv[1] << ":" << argv[2] << endl;
    IP = argv[1];
    PORT = stoi(argv[2]); // 处理char* 转 int
}

try {
    // 控制连接获取 UDP 端口
    ControlClient ctrl(IP, PORT);
    int udpPort = ctrl.RequestUdpPort();
    cout << "服务器 UDP 端口: " << udpPort << endl;

    while (true) {
        cout << "菜单:\n1-Get current time (TCP)\n2-Echo Mode (UDP)\n3-Chat (TCP)\n4-Exit the program\n请输入选项: ";
        string opt;
        getline(cin, opt);
        if (opt == "1") {
            string t = ctrl.SendTimeRequest();
            cout << "服务器时间: " << t << endl;
        }
        else if (opt == "2") {
            // UDP echo: 创建UDP socket, 向服务器发送消息并接收回显
            SOCKET u = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP);
            if (u == INVALID_SOCKET) {
                cout << "创建UDP socket失败: " << GetLastError() << endl;
                continue;
            }

            sockaddr_in serv;
            serv.sin_family = AF_INET;
            serv.sin_port = htons(udpPort);
            inet_pton(AF_INET, IP.c_str(), &serv.sin_addr);

            cout << "进入Echo模式，输入消息（输入 exit 返回菜单）: ";
            string line;
            while (true) {

```

```

        getline(cin, line);
        if (line == "exit") break;
        int sent = sendto(u, line.c_str(), (int)line.size(), 0, (SOCKADDR*)&serv,
sizeof(serv));

        if (sent == SOCKET_ERROR) {
            cout << "发送失败: " << GetLastError() << endl;
            break;
        }

        char buf[1024] = { 0 };
        int fromlen = sizeof(serv);
        int r = recvfrom(u, buf, sizeof(buf) - 1, 0, (SOCKADDR*)&serv, &fromlen);
        if (r > 0) {
            cout << "回显: " << string(buf, r) << endl;
        }

        cout << "输入消息 (输入 exit 返回菜单): ";
    }
    closesocket(u);
}

else if (opt == "3") {
    // 进入聊天, 建立一个新的 ChatClient 连接
    cout << "进入Chat, 创建聊天连接..." << endl;
    // 使用现有PORT创建聊天连接
    ChatClient chat(IP, PORT);
    chat.Start();
}

else if (opt == "4") {
    cout << "退出程序" << endl;
    break;
}

else {
    cout << "无效选项" << endl;
}

}

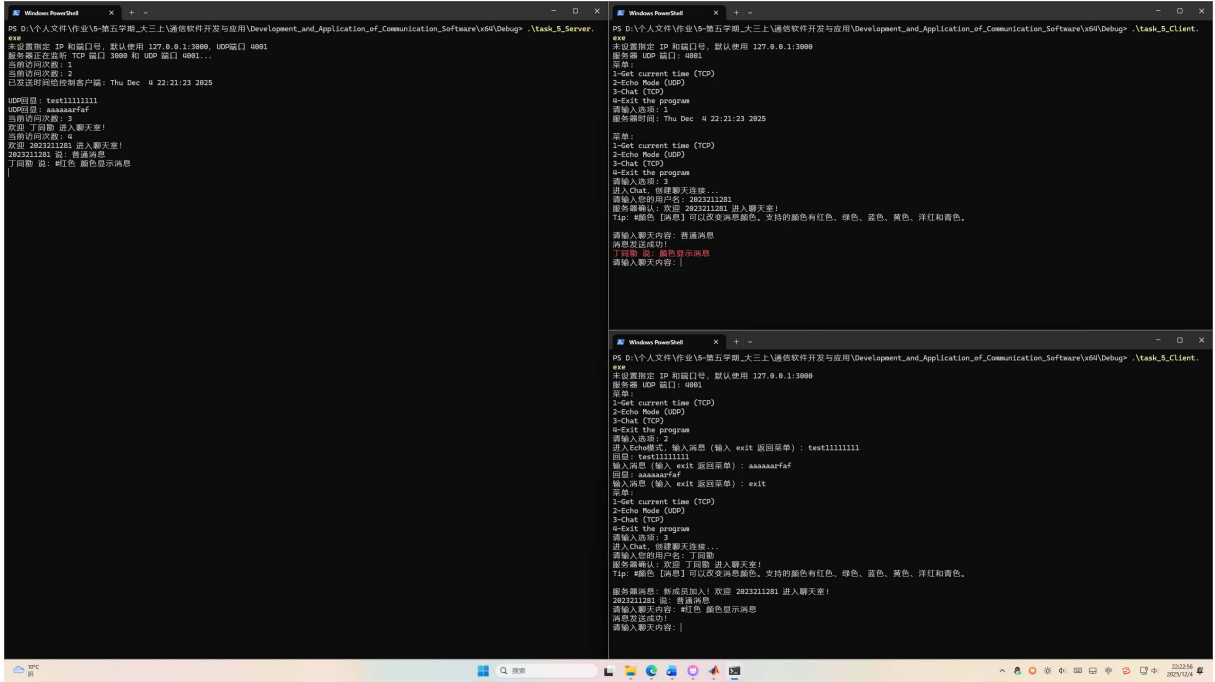
}

catch (const exception& e) {
    cout << "错误: " << e.what() << endl;
}

return 0;
}

```

四、实验结果及分析



五、实验中问题及分析

1. 控制台输入与输出的冲突（光标抢占问题）

问题描述：

在客户端聊天模式下，当用户正在输入消息但尚未发送（按下回车）时，如果收到了来自服务器转发的别人发来的消息，屏幕上的显示会变得混乱。新消息会直接打印在用户正在输入的内容后面或中间，导致界面显示错乱。

思考与分析：

- **原因：**控制台（Console）的标准输入（std::cin）和标准输出（std::cout）共享同一个光标位置。主线程阻塞在 getline 等待输入，而接收子线程随时可能调用 cout 输出接收到的消息，从而打断了当前的输入行显示。
- **分析：**这是一个典型的多线程竞争控制台资源的问题。

解决办法：

- **代码中的实现：**在 RecvMessage 函数中，使用了 \033[1K\r（ANSI 转义序列）来清除当前行并回车。

```
// 清除未使用的输入提示并打印消息
```

```
cout << "\033[1K\r";
```

```
// 打印接收到的消息...
```

```
// 重新打印提示符
```

```
cout << "请输入聊天内容：";
```

- **优化思路：**虽然代码中做了简单处理，但完美的方案需要管理输入缓冲区。更优的做法其实是使用 Windows API (ReadConsoleInput) 或 ncurses 库将屏幕分为“显示区”和“输入区”，或者在接收到消息时，先保存当前输入缓冲区的内容，清除当前行，打印新消息，然后重新把用户已输入但未发送的内容打印出来。

2. 多线程下的资源竞争（线程安全）

问题描述：

服务端若有多个客户端同时加入或退出，或者同时发送消息，可能会导致服务端崩溃，或者 clients 列表数据异常。

思考与分析：

- **原因：**std::vector<SOCKET> clients 和 std::map<SOCKET, string> clientUsernames 是全局共享

资源。当一个线程在 Start 中 push_back（增加用户），而另一个线程在 HandleChatClient 中 erase（用户退出）或遍历（广播消息）时，如果同时操作内存，会导致迭代器失效或内存访问冲突。

解决办法：

- **引入互斥锁：**代码中定义了 mutex clientsMutex。
- **实施保护：**在所有涉及读取、写入、遍历 clients 和 clientUsernames 的地方（如 Start 中的连接计数、HandleChatClient 中的加入/退出、BroadcastMessage 中的遍历），都使用了 lock_guard<mutex> lock(clientsMutex);。这利用了 RAII 机制，确保离开作用域时自动解锁，防止死锁并保证了线程安全。

3. 协议设计与“粘包”处理

问题描述：

在 TCP 传输中，服务端如何区分连接进来的是“控制客户端”还是“聊天客户端”？此外，如果客户端连续快速发送多条短消息，服务端可能会一次性收到合并后的数据（粘包）。

思考与分析：

- **区分客户端：**TCP 是流式协议，建立连接后并没有“身份”属性。需要应用层协议来握手。
- **粘包问题：**send 发送的数据会被放入内核缓冲区，TCP 协议栈可能会将多个小包合并成一个大包发送，导致 recv 一次收到多条消息，或者收到半条消息。

解决办法：

- **握手协议：**代码中设计了一个简单的握手机制。
 - 连接建立后，客户端必须发送第一条消息。
 - 服务端读取第一条消息：如果是字符串 "CTRL"，则转入 HandleControlClient；否则默认为聊天用户名，转入 HandleChatClient。
- **针对粘包的现状：**当前代码使用了固定缓冲区 buf[200] 且假设 recv 一次对应一条逻辑消息。
 - 在实际工程中，通常会在消息头部加上“长度字段”（例如前 4 个字节表示包体长度），每次先读长度，再读指定长度的内容，以彻底解决粘包问题。

4. 客户端异常退出的处理

问题描述：

如果客户端直接点击控制台右上角的“X”关闭，或者网络突然断开，服务端控制台可能会疯狂打印空消息或直接报错退出。

思考与分析：

- **原因：**当 TCP 连接断开时，recv 函数会返回 0（正常关闭）或 -1（错误，SOCKET_ERROR）。如果代码中没有正确判断返回值，依然尝试处理 buffer，就会出错。

解决办法：

- **健壮的返回值检查：**

在 HandleChatClient 的 recv 循环中：

```
ret = recv(clientSocket, bufMsg, sizeof(bufMsg) - 1, 0);
if (ret > 0) {
    // 正常处理
}
else {
    // ret == 0 表示断开，< 0 表示错误
    // 跳出循环，执行清理工作（移除用户、关闭Socket）
}
```

代码中正是利用了 do...while(ret > 0) 结构，确保了连接断开时能正确退出循环并清理资源。

5. 自定义颜色协议的解析逻辑

问题描述：

客户端想要实现“#红色 消息内容”变色功能，但解析逻辑容易出错，比如用户输入“#红色”后面没有空格，或者输入不支持的颜色，或者用户名含有空格，打乱了颜色解析的结构。

思考与分析：

- **分析：**这涉及到字符串的解析（Parsing）。需要定位特定字符（'#'）和分隔符（空格）。

解决办法：

- **逻辑实现：**
 1. 查找 # 的位置。
 2. 查找紧随其后的空格位置。
 3. 截取中间的子串作为 colorCode。
 4. 截取空格后的内容作为消息体。
 5. 控制用户名不能含有空格。
- **容错处理：**代码中加入了 `receivedMessage.size() > 5` 和 `spacePos2 != string::npos` 等判断条件，防止越界访问导致的程序崩溃（Access Violation）。如果格式不匹配，则回退到普通消息显示。

六、心得体会

通过本次基于 Socket 的网络编程实验，我设计并实现了一个集聊天、时间获取和 UDP 回显功能于一体的客户端/服务端系统。这不仅加深了我对计算机网络底层原理的理解，也锻炼了我 C++ 多线程环境下解决实际问题的能力。以下是我在实验过程中的主要收获和体会：

1. 深入理解 TCP 与 UDP 的异同与应用场景

在代码设计中，我同时使用了 TCP 和 UDP 两种协议。

- **TCP 的可靠性：**在聊天和控制命令（如获取时间）模块中，我选择了 TCP。实验让我深刻体会到 TCP 面向连接、可靠传输的特性。对于聊天消息，顺序和完整性至关重要，不能出现丢包或乱序，因此 TCP 是不二之选。
- **UDP 的无连接特性：**在回显（Echo）模块中，我使用了 UDP。这让我理解了 UDP 的轻量级和无状态特性。虽然它不保证送达，但在实现简单的“发后即忘”或实时性要求高的功能时，UDP 更加高效。
- **混合架构的实践：**在一个端口监听 TCP，另一个端口监听 UDP，这种混合监听的设计模拟了真实服务器处理多种业务的场景，拓宽了我的视野。

2. 多线程编程与并发控制的挑战

本次实验最大的难点在于**并发处理**。

- **线程分离：**服务端必须同时处理多个客户端的连接，还要监听 UDP 和 TCP 请求。我学会了使用 C++11 的 `std::thread` 并结合 `.detach()` 来实现非阻塞的并发处理，确保主线程可以持续接受新连接。
- **资源竞争与互斥锁：**在维护在线客户端列表（`clients` 和 `clientUsernames`）时，我深刻意识到

了“线程安全”的重要性。多个线程同时写入或删除列表会导致数据竞争甚至程序崩溃。通过引入 `std::mutex` 和 `std::lock_guard`, 我学会了如何利用 RAII 机制保护共享资源, 避免了死锁和数据不一致的问题。

3. 应用层协议的设计与实现

Socket 只是传输数据的管道, 如何“理解”数据是应用层的责任。

- **握手协议:** 为了在一个 TCP 端口上区分“聊天用户”和“控制指令”, 我设计了一个简单的握手逻辑 (首条消息发送 "CTRL" 或 "用户名")。这让我明白了标准协议 (如 HTTP、FTP) 头部设计的必要性。
- **自定义文本协议:** 在实现颜色功能时, 我设计了形如 `#红色 消息` 的文本协议。编写解析逻辑的过程锻炼了我的字符串处理能力, 也让我意识到协议设计的健壮性非常重要 (例如处理空格缺失、格式错误等边界情况)。

4. 阻塞 I/O 与用户体验的平衡

在客户端开发中, 我遇到了 `cin` (输入阻塞) 和 `recv` (接收阻塞) 的冲突问题。

- 如果单线程运行, 用户在输入时将无法接收消息, 体验极差。
- 通过将接收逻辑放入子线程, 实现了发送与接收的分离。
- 同时, 我也学会了如何通过控制台转义字符 (`\033[1K\r`) 来优化显示效果, 解决光标抢占问题。这让我明白, 网络编程不仅仅是后端的数据交换, 前端的交互体验同样需要精细的逻辑控制。

5. 健壮性与异常处理

实验初期, 客户端强行断开连接常导致服务端崩溃。通过调试, 我学会了判断 `recv` 的返回值:

- 区分 0 (正常断开) 和 `SOCKET_ERROR` (异常)。
- 在检测到连接断开时, 必须及时清理资源 (从 `vector` 中移除 `Socket`、关闭句柄), 否则会导致资源泄露或无效的广播。这培养了我编写“防御性代码”的意识。

本次实验不仅是一次代码实现的练习, 更是一次系统工程的实践。从底层的 `Socket API` 调用, 到中间的协议设计, 再到上层的多线程同步与 UI 交互, 每一个环节都环环相扣。通过解决“粘包”、“中文乱码”、“线程同步”等具体问题, 我从理论走向了实践, 真正掌握了网络编程的核心思想。在未来的学习中, 我希望能进一步探索 I/O 复用模型 (如 `Select/Epoll`) 以应对更高并发的场景。